Recursive Functions, More about functions

Sudeep K S, Mahindra University (October 2026)

A **recursive function** is a function that calls itself at least once.

Example:

```
int sum_upto_n(int n)
{
    if (n < 1) {
       printf("n should be greater than or equal to 1\n");
       return 0;
    }

    // the stop condition, there is no further call
    if (n == 1)
       return 1;

    return (sum_upto_n(n - 1) + n);
}
```

When it is called with n = 5, for instance, it would first call the same function with n = 4. In turn, it calls the same function with n = 3, .. and so on. When it is called with n = 1, the 'if' condition holds true and the function returns the value 1. At the higher level, it adds 2 to this value 1 and gets sum_upto_n(2) = sum_upto_n(1) + 2 = 1 + 2 = 3. Similarly, at one more level higher, it calculates sum_upto_n(3) as sum_upto_n(2) + 2 = 3 + 3 = 6. This goes on, till at the higher-most level, it returns 1 + 2 + 3 + 4 + 5 = 15 as the answer. See the exact sequence below.

`sum_upto_n(5)` calls `sum_upto_n(4)`

`sum_upto_n(4)` calls `sum_upto_n(3)`

`sum_upto_n(3)` calls `sum_upto_n(2)`

`sum_upto_n(2)` calls `sum_upto_n(1)`

`sum_upto_n(1)` returns 1

`sum_upto_n(2)` returns 3, which is `sum_upto_n(1) + 2` (n is 2)

`sum_upto_n(3)` returns 6, which is `sum_upto_n(2) + 3`

`sum_upto_n(4)` returns 10, which is `sum_upto_n(3) + 4`

`sum_upto_n(5)` returns 15, which is `sum_upto_n(4) + 5`

Another simple and natural example is that of factorial. Let us assume it is called with n ≥ 0.

```
int factorial(int n)
{
    // the stop condition (base case)
    if (n == 0)
       return 1;

    return (factorial(n - 1) * n);
}
```

Note that this does not explicitly define factorial of n as 1 x 2 x … x n, but that is what eventually happens when this function is called. When it is called with n = 10, for instance, it would first call the same function with n = 9. That in turn will call the same function with n = 8, .. and so on.

It ends when n becomes 0, where it hits the stop condition and returns 1. At the higher level, it multiplies this value with 1 and returns the value of factorial(1) as `factorial(0) * 1`, (remember that n = 1 and n – 1 is 0), which is 1 x 1. This goes on, till at the highest level, it computes factorial(10) as factorial(9) x 10, as (1 x 1 x 2 x … x 9) x 10.

Below is GCD function defined in a recursive matter, assuming that a ≥ b. **Exercise:** Write down all the calls to this function when we call (i) gcd(18, 12) and (ii) gcd(100, 35).

```
int gcd(int a, int b)
{
    // the stop condition
    if (b == 0)
       return a;

    return(gcd(b, a % b)); // recursive call to same function
}
```

A function to find n^{th} Fibonacci number (sequence 0, 1, 2, 3, 5, 8, 13, 21, …) is given below.

```
int fibonacci(int n)
{
   if (n == 1)
      return 0;

   if (n == 2)
      return 1;

   return (fibonacci(n - 1) + fibonacci(n - 2));
}
```

Note that two stop conditions are required here, because fibonacci(n) depends on two previous values, fibonacci(n – 1) and fibonacci(n -2).

Exercise: Can this be done without using recursion? (Hint: Use a for loop or a while loop)

One function calling another

Just like calling the same function from a function, we can call a different function from one function. Below is such an example – a function that prints all prime numbers between a and b, when we give two integer numbers a and b. It calls the prime() function we defined earlier.

```
// function to print all prime numbers between a and b
// Example: when a = 10, b = 15:
// Expected output:
//    prime numbers between 10 and 15 are:
//    11, 13
void printprimes(int a, int b)
{
    int i;

    if (a > b) {
       printf("first number should be smaller\n");
       return;
    }

    // now b >= a, second number is same or is bigger compared to a
    printf("prime numbers between %d and %d are:\n", a, b);
    for (i = a; i <= b; i++)
    {
        if (prime(i))
           printf("%d ", i);  // if i is prime, print the value of i
    }

    printf("\n"); // add a line break when the list ends
    return;
}
```

A slightly more complex function: **To sort a given array of length n**.

The logic is to compare each consecutive pair of elements starting from the leftmost pair and going right, and swap them if they are not in order. The last pair that gets compared would be elements at index n – 2 and n – 1. Once you reach the right end, the highest element comes to the rightmost index (that is n – 1) in the array. We can then repeat it excluding the last element (it would end with comparison of index n – 3 and n – 2) , and so on.

This sorting technique is called 'bubble sort'.

```
void sortarray(int marks[], int n)
{
   int i, j, temp;
   for (j = n - 1; j >= 1; j--) // the rightmost index right now
   {
      for (i = 0; i < j; i++)
      {
        if (marks[i] > marks[i + 1]) // swap if they are not in order
        {
          temp = marks[i];
          marks[i] = marks[i + 1];
          marks[i + 1] = temp;
        }
      }
   }
}
```

It turns out that this also can be made recursive. Logic remains the same. Try it yourself. See next page for a solution.

```
void sortarray(int marks[], int n)
{
    int i, temp;

    if (n == 1) return; // no need to do anything if n is 1

    for (i = 0; i < n - 1; i++)
    {
        if (marks[i] > marks[i + 1]) // swap if not in order
        {
          temp = marks[i];
          marks[i] = marks[i + 1];
          marks[i + 1] = temp;
        }
    }
    sortarray(marks, n - 1); // recursive call on n – 1 elements
}
```